claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a33bc755df8ab1511.jsonl`
- SHA-256: `cd0a6459e203b6d2997fa53b658fde85febc95883786782d5036384a12f6f700`
- Source modified: `2026-06-10T16:28:34+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:25:50.944Z)

```
You are an adversarial verifier. A code reviewer audited the repo at
C:/Users/avidu/Projects/gmail-attachments-to-gdrive and reported this finding:

TITLE: GmailApp.getMessageById throws on deleted messages, wedging incremental sync until the
cursor expires
SEVERITY CLAIMED: high
FILE: src/gmailHistory.ts (line 70-77)
DESCRIPTION: threadsForMessageIds assumes a falsy return for a message that no longer exists
('Messages that can no longer be fetched ... are skipped'). In the real GAS runtime
GmailApp.getMessageById throws an exception for an id that is invalid or permanently deleted; it
does not return null (the unit tests stub it to return null, which is why this passes). A
message recorded in history and then deleted/purged before the run (user deletes junk right
after receipt ? common at a 20-minute trigger cadence) makes every incremental run throw. The
exception propagates to the catch in runSyncOnce BEFORE state.setHistoryId is called, so the
cursor never advances and every subsequent run re-fetches the same dead id and throws again. The
wedge persists until the stored historyId itself expires (typically a week or more), during
which no new attachments sync; if retention exceeds 30 days, the eventual 30-day rewind also
leaves a permanent gap.
EVIDENCE QUOTED: for (const id of messageIds) {
    const message = GmailApp.getMessageById(id);
    if (!message) {
      continue;
    }
SUGGESTED FIX: Wrap the per-id fetch in try/catch and skip ids that throw (log at debug level).
Add a test whose fake getMessageById throws, mirroring the real runtime contract.

Your job is to try to REFUTE it. Your lens: correctness: trace the actual code path the finding
describes and check the claimed behavior is what the code really does (read the file and its
callers/tests).
Read the actual code in the repo (and tests/config where relevant). If after reading you cannot
confirm the problem is real, set isReal=false. If it is real but over/under-stated, set
adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/,
testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL
financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file
existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess
sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers,
balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account
number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.
Structured output only.
```

2. user (2026-06-10T16:25:59.648Z)

```
import {
  collectAddedMessageIds,
  isHistoryExpired,
  nextHistoryId,
  type HistoryListResponse,
} from "./historySync";
import { logDebug } from "./logger";

type GmailThread = GoogleAppsScript.Gmail.GmailThread;

/** A bounded number of history pages to walk in a single run. */
const MAX_HISTORY_PAGES = 20;

/** The mailbox's current historyId, used to seed incremental sync. */
export function getCurrentHistoryId(): string | null {
  const profile = Gmail.Users.getProfile("me");
  return profile.historyId ?? null;
}

/** What an incremental scan yields: new message ids and the id to store next. */
export interface IncrementalChanges {
  messageIds: string[];
  historyId: string;
}

/**
 * Lists messages added since `startHistoryId` via the Gmail advanced service,
 * paging through results. Returns null when the history has expired (the caller
 * must fall back to a full sync). Only `messageAdded` events are requested.
 */
export function listAddedSince(
  startHistoryId: string,
): IncrementalChanges | null {
  const responses: HistoryListResponse[] = [];
  let pageToken: string | undefined;
  try {
    for (let page = 0; page < MAX_HISTORY_PAGES; page++) {
      const response = Gmail.Users.History.list("me", {
        startHistoryId,
        historyTypes: ["messageAdded"],
        pageToken,
      }) as HistoryListResponse;
      responses.push(response);
      if (!response.nextPageToken) {
        break;
      }
      pageToken = response.nextPageToken;
    }
  } catch (e) {
    if (isHistoryExpired(e)) {
      logDebug("History id expired; full sync required.");
      return null;
    }
    throw e;
  }
  return {
    messageIds: collectAddedMessageIds(responses),
    historyId: nextHistoryId(responses, startHistoryId),
  };
}

/**
 * Resolves the distinct Gmail threads containing the given message ids. Threads
```

```
64       * are de-duplicated by id so a multi-message thread is processed once. Messages
65       * that can no longer be fetched (deleted since the change was recorded) are
66       * skipped.
67       */
68      export function threadsForMessageIds(messageIds: string[]): GmailThread[] {
69        const threads = new Map<string, GmailThread>();
70        for (const id of messageIds) {
71          const message = GmailApp.getMessageById(id);
72          if (!message) {
73            continue;
74          }
75          const thread = message.getThread();
76          threads.set(thread.getId(), thread);
77        }
78        return Array.from(threads.values());
79      }
80
```

3. user (2026-06-10T16:26:00.405Z)

```
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\config.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\contentTypes.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\drive.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\entry.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\historySync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\logger.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\naming.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\processed.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\query.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\quota.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\stats.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\types\gmailAdvanced.d.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\ui.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\auditability.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\contentTypes.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\historySync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\naming.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notify.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\processed.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\quota.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\setup.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\stats.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts
```

4. user (2026-06-10T16:26:13.329Z)

```
1       import {
2         ENABLE_INCREMENTAL_SYNC,
3         MAX_FILES_ADD_PER_DAY,
4         MAX_RUNTIME_MS,
5         MAX_SYNC_TRIGGERS,
6         ONE_DAY,
7         ONE_MINUTE,
8         SIX_HOURS,
9         THIRTY_DAYS,
10      } from "./config";
```

```
import { updateDrive, type UpdateResult } from "./drive";
import {
  getCurrentHistoryId,
  listAddedSince,
  threadsForMessageIds,
} from "./gmailHistory";
import { logDebug, logError } from "./logger";
import { maybeSendDailySummary } from "./notify";
import { addProcessedId } from "./processed";
import { buildSearchQuery } from "./query";
import { hasTime, isSameCalendarDay } from "./quota";
import { dayKey, recordRun } from "./stats";
import * as state from "./state";

const SYNC_HANDLER = "syncUserDrives";

/** Schedules a one-shot run of syncUserDrives `afterMs` from now. */
export function newSyncTrigger(afterMs: number): void {
  ScriptApp.newTrigger(SYNC_HANDLER).timeBased().after(afterMs).create();
}

/** All currently-installed triggers for the sync handler. */
function getSyncTriggers(): GoogleAppsScript.Script.Trigger[] {
  return ScriptApp.getProjectTriggers().filter(
    (t) => t.getHandlerFunction() === SYNC_HANDLER,
  );
}

/** Removes every project trigger (used when the user re-runs setup). */
export function deleteAllTriggers(): void {
  for (const trigger of ScriptApp.getProjectTriggers()) {
    ScriptApp.deleteTrigger(trigger);
  }
}

function deleteTriggers(triggers: GoogleAppsScript.Script.Trigger[]): void {
  for (const trigger of triggers) {
    ScriptApp.deleteTrigger(trigger);
  }
}

/** Defense-in-depth: never let sync triggers exceed the cap (limit is 20). */
function pruneSyncTriggers(max: number): void {
  const triggers = getSyncTriggers();
  for (let i = max; i < triggers.length; i++) {
    ScriptApp.deleteTrigger(triggers[i]);
  }
}

/**
 * The background worker. A per-user lock ensures only one run executes at a time
 * for THIS user, so concurrent triggers cannot stack file copies or trigger
 * generations. It is deliberately a user lock, not a script lock: the web app
 * runs as the accessing user (executeAs USER_ACCESSING), so a script-wide lock
 * would serialise every user's sync into one global queue.
 */
export function syncUserDrives(): void {
  const lock = LockService.getUserLock();
  if (!lock.tryLock(0)) {
    logDebug("Another sync run is in progress; skipping.");
    return;
  }
  try {
    runSyncOnce();
  } finally {
```

```
        // Bound the trigger count regardless of which path we took, then release.
        pruneSyncTriggers(MAX_SYNC_TRIGGERS);
        lock.releaseLock();
      }
    }

    /**
     * A single sync pass:
     *   1. reschedules a fan-out of backup triggers (self-healing if one is lost),
     *   2. syncs forward in 30-day windows until it runs out of time, hits the
     *      daily file cap, or fully catches up,
     *   3. persists progress and schedules the next run.
     */
    function runSyncOnce(): void {
      const today = new Date();

      // On the first run of a new calendar day, summarise the previous day.
      maybeSendDailySummary(today);

      // Recover state if the debug flag was cleared (e.g. properties were wiped).
      const debug = state.getDebug();
      if (debug == null || debug === "0") {
        state.setDebug("1");
        state.setLastSynced(new Date(today.getTime() - 365 * ONE_DAY).toJSON());
      }

      // Create the backup triggers BEFORE deleting the current ones, so a failure
      // here can never leave us with no scheduled run.
      const previousTriggers = getSyncTriggers();
      try {
        newSyncTrigger(60 * ONE_MINUTE);
        newSyncTrigger(SIX_HOURS);
        newSyncTrigger(2 * SIX_HOURS);
        newSyncTrigger(4 * SIX_HOURS);
        newSyncTrigger(8 * SIX_HOURS);
        newSyncTrigger(28 * SIX_HOURS);
      } catch (e) {
        // Could not install new triggers; leave the old ones and try again later.
        logError("Could not install triggers: " + e);
        return;
      }
      deleteTriggers(previousTriggers);

      const filesAddedToday = state.getFilesAddedToday();
      let added = filesAddedToday.added;
      const lastCountDate = new Date(Date.parse(filesAddedToday.date));

      if (isSameCalendarDay(lastCountDate, today) && added >= MAX_FILES_ADD_PER_DAY) {
        logDebug("Used up all quota for today. Can't add more files.");
        newSyncTrigger(60 * ONE_MINUTE);
        return;
      }
      if (!isSameCalendarDay(lastCountDate, today)) {
        added = 0;
      }

      const structure = state.getFolderStructure();
      let lastSynced = state.getLastSynced();

      // Load the dedup set once; thread it through the run and persist at the end.
      let processedIds = state.getProcessedIds();
      const processedSet = new Set(processedIds);

      // This run's contribution to the day's stats.
      const addedAtStart = added;
```

```
      let runErrors = 0;
      let runLastError = "";

      // Folds one updateDrive result into this run's accumulators.
      const applyResult = (result: UpdateResult): void => {
        added += result.addedFiles;
        runErrors += result.errors;
        if (result.lastError) {
          runLastError = result.lastError;
        }
        for (const id of result.processedIds) {
          if (!processedSet.has(id)) {
            processedSet.add(id);
            processedIds = addProcessedId(processedIds, id);
          }
        }
      };

      try {
        // Incremental mode is opt-in (ENABLE_INCREMENTAL_SYNC) so it rolls out
        // gradually over the proven date-window backfill. A stored historyId only
        // exists once a backfill has caught up while the flag was on.
        const historyId = ENABLE_INCREMENTAL_SYNC ? state.getHistoryId() : null;
        if (historyId) {
          // Incremental: process only messages added since the stored historyId.
          const changes = listAddedSince(historyId);
          if (changes == null) {
            // History expired: clear the cursor and re-scan a bounded recent window
            // (the next catch-up re-captures a fresh historyId). Overlap is safe ?
            // dedup by message id and the Drive filename guard prevent re-copying.
            state.clearHistoryId();
            lastSynced = new Date(today.getTime() - THIRTY_DAYS).toJSON();
            logDebug("History expired; reverting to date-window sync.");
          } else {
            if (changes.messageIds.length > 0 && added < MAX_FILES_ADD_PER_DAY) {
              const threads = threadsForMessageIds(changes.messageIds);
              if (threads.length > 0) {
                applyResult(
                  updateDrive(
                    threads,
                    structure,
                    MAX_FILES_ADD_PER_DAY - added,
                    processedSet,
                  ),
                );
              }
            }
            // Advance the cursor so the next run only sees newer changes.
            state.setHistoryId(changes.historyId);
          }
        } else {
          // Backfill: walk forward in 30-day windows until caught up to today.
          // Capture the mailbox historyId BEFORE processing so mail arriving
          // mid-run is still picked up by the first incremental scan.
          const candidateHistoryId = ENABLE_INCREMENTAL_SYNC
            ? getCurrentHistoryId()
            : null;
          let fullSynced = false;
          while (
            hasTime(new Date(), today, MAX_RUNTIME_MS) &&
            added < MAX_FILES_ADD_PER_DAY &&
            !fullSynced
          ) {
            const maxToAdd = MAX_FILES_ADD_PER_DAY - added;
            const search = buildSearchQuery(lastSynced, new Date());
```

```
            logDebug("Query: " + search.query + "  files asking: " + maxToAdd);

            const threads = GmailApp.search(search.query);
            if (threads.length > 0) {
              applyResult(updateDrive(threads, structure, maxToAdd, processedSet));
            }

            fullSynced = search.isToday;
            if (added < MAX_FILES_ADD_PER_DAY) {
              lastSynced = fullSynced
                ? today.toJSON()
                : new Date(Date.parse(lastSynced) + THIRTY_DAYS).toJSON();
            }
          }
          // Caught up: switch future runs to the cheaper incremental path.
          if (candidateHistoryId && fullSynced && added < MAX_FILES_ADD_PER_DAY) {
            state.setHistoryId(candidateHistoryId);
            logDebug("Backfill complete; stored historyId " + candidateHistoryId);
          }
        }
      } catch (e) {
        logError("Some error in execution: " + e);
        state.setLastError(String(e));
        runErrors++;
        runLastError = String(e);
      }

      // Fold this run into the day's accumulator.
      const dayStats = recordRun(state.getRunStats(dayKey(today)), {
        filesAdded: added - addedAtStart,
        errors: runErrors,
        lastError: runLastError,
      });
      state.setRunStats(dayStats);

      state.setProcessedIds(processedIds);
      state.setLastSynced(lastSynced);
      state.setFilesAddedToday({ added, date: today.toJSON() });
      newSyncTrigger(20 * ONE_MINUTE);
    }

```

5. user (2026-06-10T16:26:13.618Z)

```
import {
  getCurrentHistoryId,
  listAddedSince,
  threadsForMessageIds,
} from "../src/gmailHistory";

const g = globalThis as unknown as Record<string, unknown>;

describe("getCurrentHistoryId", () => {
  it("returns the profile's historyId", () => {
    g.Gmail = {
      Users: { getProfile: () => ({ historyId: "99" }), History: { list: jest.fn() } },
    };
    expect(getCurrentHistoryId()).toBe("99");
  });

  it("returns null when the profile has no historyId", () => {
    g.Gmail = {
      Users: { getProfile: () => ({}), History: { list: jest.fn() } },
    };
    expect(getCurrentHistoryId()).toBeNull();
```

```
  });
});

describe("listAddedSince", () => {
  it("pages through results, returning collected ids and the latest historyId", () => {
    const pages = [
      {
        history: [{ messagesAdded: [{ message: { id: "m1" } }] }],
        historyId: "100",
        nextPageToken: "p2",
      },
      {
        history: [{ messagesAdded: [{ message: { id: "m2" } }] }],
        historyId: "200",
      },
    ];
    let call = 0;
    const list = jest.fn(() => pages[call++]);
    g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };

    expect(listAddedSince("50")).toEqual({
      messageIds: ["m1", "m2"],
      historyId: "200",
    });
    expect(list).toHaveBeenCalledTimes(2);
  });

  it("returns null when the start historyId has expired", () => {
    const list = jest.fn(() => {
      throw new Error("call failed with error: 404 not found");
    });
    g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
    expect(listAddedSince("1")).toBeNull();
  });

  it("rethrows errors that are not history expiry", () => {
    const list = jest.fn(() => {
      throw new Error("backend error 500");
    });
    g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
    expect(() => listAddedSince("1")).toThrow("500");
  });
});

describe("threadsForMessageIds", () => {
  it("resolves distinct threads and skips messages that no longer exist", () => {
    const threadA = { getId: () => "tA" };
    const byId: Record<string, { getThread: () => { getId: () => string } }> = {
      a: { getThread: () => threadA },
      b: { getThread: () => threadA }, // same thread as a
      c: { getThread: () => ({ getId: () => "tC" }) },
    };
    g.GmailApp = { getMessageById: (id: string) => byId[id] ?? null };

    const threads = threadsForMessageIds(["a", "b", "c", "gone"]);
    expect(threads.map((t) => (t as { getId: () => string }).getId()).sort()).toEqual(
      ["tA", "tC"],
    );
  });
});

```

6. user (2026-06-10T16:27:11.092Z)

```
/*
```

```
 * The MIT License (MIT)
 * Copyright (c) 2013 Avi Dullu
 */

/** Top-level Drive folder that holds every synced attachment. */
export const GMAIL_ATTACHMENT_FOLDER = "GmailAttachments";

/** Public project / support page shown in the setup UI. */
export const SUPPORT_URL =
  "https://github.com/avidullu/gmail-attachments-to-gdrive";

/** Privacy policy shown in the setup UI (and the OAuth consent screen). */
export const PRIVACY_POLICY_URL =
  "https://github.com/avidullu/gmail-attachments-to-gdrive/blob/master/PRIVACY.md";

/**
 * Apps Script enforces a daily quota on file creation. We cap ourselves below
 * it so a single run can never exhaust the account's allowance.
 */
export const MAX_FILES_ADD_PER_DAY = 240;

// Time constants, in milliseconds.
export const ONE_MINUTE = 60_000;
export const ONE_DAY = 86_400_000;
export const THIRTY_DAYS = 2_592_000_000;
export const ONE_YEAR = 31_622_400_000;
export const SIX_HOURS = 21_600_000;

/**
 * Wall-clock budget for a single sync execution. Apps Script kills consumer
 * scripts at 6 minutes, so we stop at 5 to leave margin for the final
 * bookkeeping and trigger scheduling.
 */
export const MAX_RUNTIME_MS = 5 * ONE_MINUTE;

/** Defensive upper bound on how many sync triggers may exist at once (limit is 20). */
export const MAX_SYNC_TRIGGERS = 12;

// Keys used in PropertiesService.getUserProperties().
export const PROP_FOLDER = "f";
export const PROP_LAST_SYNCED = "l";
export const PROP_FILES_ADDED_TODAY = "fa";
export const PROP_DEBUG = "dbg";
export const PROP_LAST_ERROR = "err";
export const PROP_PROCESSED_IDS = "pid";
export const PROP_RUN_STATS = "stats";
export const PROP_SUMMARY_LAST_SENT = "sumsent";
export const PROP_HISTORY_ID = "hid";

/** Flip to true to emit verbose Logger output during development. */
export const ENABLE_DEBUGGING = false;

/**
 * Hybrid incremental sync. OFF by default so the proven date-window backfill
 * remains the sole code path; flip to true to roll out the cheaper Gmail
 * `history.list` steady state. When on, once a backfill catches up the current
 * mailbox `historyId` is captured and subsequent runs process only newly-added
 * messages, falling back to a date-window re-scan if the id expires (HTTP 404).
 * Requires the Gmail advanced service (declared in appsscript.json); no extra
 * OAuth scope ? `gmail.readonly` already covers history reads.
 */
export const ENABLE_INCREMENTAL_SYNC = false;

/**
 * Daily summary email. OFF by default ? enabling it requires the
```

```
67       * `script.send_mail` OAuth scope (a re-consent on next deploy). When enabled,
68       * set SUMMARY_EMAIL_RECIPIENT to the destination address (empty = the
69       * effective user's own address).
70       */
71      export const ENABLE_SUMMARY_EMAIL = false;
72      export const SUMMARY_EMAIL_RECIPIENT = "";
73
```

7. user (2026-06-10T16:27:12.097Z)

```
<tool_use_error>Path does not exist: C:\Users\avidu\Projects\gmail-attachments-to-
gdrive\node_modules\@types\google-apps-script. Note: your current working directory is
C:\Users\avidu\Projects\muneem.</tool_use_error>
```

8. user (2026-06-10T16:27:19.439Z)

```
No files found
```

9. user (2026-06-10T16:27:19.752Z)

```
1       {
2         "name": "gmail-attachments-to-gdrive",
3         "version": "2.0.0",
4         "description": "Google Apps Script that syncs Gmail attachments into Google Drive.",
5         "license": "MIT",
6         "private": true,
7         "scripts": {
8           "build": "node esbuild.config.mjs",
9           "typecheck": "tsc --noEmit",
10          "lint": "eslint src tests",
11          "test": "jest",
12          "test:coverage": "jest --coverage",
13          "push": "npm run build && clasp push",
14          "deploy": "npm run build && clasp deploy"
15        },
16        "devDependencies": {
17          "@google/clasp": "^2.4.2",
18          "@types/google-apps-script": "^1.0.83",
19          "@types/jest": "^29.5.12",
20          "@types/node": "^20.12.7",
21          "esbuild": "^0.21.5",
22          "eslint": "^9.5.0",
23          "jest": "^29.7.0",
24          "ts-jest": "^29.1.4",
25          "typescript": "^5.4.5",
26          "typescript-eslint": "^8.0.0"
27        }
28      }
29
```

10. user (2026-06-10T16:27:55.666Z)

```
1       // Enable the opt-in incremental path for this suite only.
2       jest.mock("../src/config", () => ({
3         ...jest.requireActual("../src/config"),
4         ENABLE_INCREMENTAL_SYNC: true,
5       }));
6
7       import {
8         ONE_DAY,
9         PROP_DEBUG,
10        PROP_FILES_ADDED_TODAY,
11        PROP_FOLDER,
12        PROP_HISTORY_ID,
```

```
  PROP_LAST_SYNCED,
  PROP_PROCESSED_IDS,
} from "../src/config";
import { syncUserDrives } from "../src/sync";

type Store = Record<string, string>;
const g = globalThis as unknown as Record<string, unknown>;
const nowIso = () => new Date().toJSON();

function fakeProps(initial: Store) {
  const store: Store = { ...initial };
  return {
    store,
    getProperty: (k: string) => (k in store ? store[k] : null),
    setProperty: (k: string, v: string) => {
      store[k] = String(v);
    },
    deleteProperty: (k: string) => {
      delete store[k];
    },
    deleteAllProperties: () => {
      for (const k of Object.keys(store)) delete store[k];
    },
  };
}

function fakeScriptApp() {
  const triggers: { getHandlerFunction: () => string }[] = [];
  return {
    newTrigger: (handler: string) => ({
      timeBased: () => ({
        after: () => ({
          create: () => triggers.push({ getHandlerFunction: () => handler }),
        }),
      }),
    }),
    getProjectTriggers: () => [...triggers],
    deleteTrigger: () => undefined,
  };
}

function iter<T>(items: T[]) {
  let i = 0;
  return { hasNext: () => i < items.length, next: () => items[i++] };
}

function installBase(props: ReturnType<typeof fakeProps>) {
  g.PropertiesService = { getUserProperties: () => props };
  g.ScriptApp = fakeScriptApp();
  g.LockService = {
    getUserLock: () => ({ tryLock: () => true, releaseLock: jest.fn() }),
  };
  g.GmailApp = { search: jest.fn(() => []), getMessageById: jest.fn() };
}

function baseProps(overrides: Store = {}) {
  return fakeProps({
    [PROP_DEBUG]: "1",
    [PROP_FOLDER]: "byContentType",
    [PROP_LAST_SYNCED]: nowIso(),
    [PROP_FILES_ADDED_TODAY]: JSON.stringify({ added: 0, date: nowIso() }),
    ...overrides,
  });
}

```

```
    describe("syncUserDrives ? incremental mode", () => {
      it("processes messages added since the stored historyId and advances it", () => {
        const props = baseProps({ [PROP_HISTORY_ID]: "100" });
        installBase(props);

        // A Drive head folder that accepts one file.
        const head = {
          getName: () => "GmailAttachments",
          getFolders: () => iter([]),
          createFolder: () => head,
          getFilesByName: () => iter([]),
          createFile: () => ({ setDescription: () => undefined }),
        };
        g.DriveApp = { getFolders: () => iter([]), createFolder: () => head };

        const message = {
          getId: () => "m1",
          getFrom: () => "alice@example.com",
          getDate: () => new Date(2012, 5, 6),
          getAttachments: () => [
            {
              getContentType: () => "application/pdf",
              getName: () => "report.pdf",
              copyBlob: () => ({ setName: () => undefined }),
            },
          ],
        };
        (g.GmailApp as { getMessageById: jest.Mock }).getMessageById = jest.fn(
          () => ({ getThread: () => ({ getId: () => "t1", getMessages: () => [message] })
}),
        );
        g.Gmail = {
          Users: {
            getProfile: jest.fn(),
            History: {
              list: jest.fn(() => ({
                history: [{ messagesAdded: [{ message: { id: "m1" } }] }],
                historyId: "200",
              })),
            },
          },
        };

        syncUserDrives();

        // Never falls back to a date-window search while the history is valid.
        expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
        expect(props.store[PROP_HISTORY_ID]).toBe("200");
        expect(JSON.parse(props.store[PROP_PROCESSED_IDS])).toContain("m1");
      });

      it("clears the cursor and rewinds lastSynced when the history expired", () => {
        const props = baseProps({ [PROP_HISTORY_ID]: "1" });
        installBase(props);
        g.Gmail = {
          Users: {
            getProfile: jest.fn(),
            History: {
              list: jest.fn(() => {
                throw new Error("call failed with error: 404 not found");
              }),
            },
          },
        };

```

```
142         syncUserDrives();
143
144         expect(PROP_HISTORY_ID in props.store).toBe(false);
145         // lastSynced rewound to roughly 30 days ago for the recovery re-scan.
146         const lastSynced = Date.parse(props.store[PROP_LAST_SYNCED]);
147         expect(Date.now() - lastSynced).toBeGreaterThan(29 * ONE_DAY);
148         // No history ? no incremental processing this run; backfill resumes next run.
149         expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
150       });
151
152       it("captures the mailbox historyId once backfill catches up", () => {
153         // No stored historyId ? backfill mode; lastSynced=now means we are already
154         // caught up, so the run captures the current historyId for next time.
155         const props = baseProps();
156         installBase(props);
157         g.Gmail = {
158           Users: {
159             getProfile: jest.fn(() => ({ historyId: "555" })),
160             History: { list: jest.fn() },
161           },
162         };
163
164         syncUserDrives();
165
166         expect((g.GmailApp as { search: jest.Mock }).search).toHaveBeenCalled();
167         expect(props.store[PROP_HISTORY_ID]).toBe("555");
168       });
169     });
170
```

11. user (2026-06-10T16:28:28.780Z)

Structured output provided successfully

12. assistant (2026-06-10T16:28:34.253Z)

Verification complete. The finding is real (code path and cursor-wedge mechanism fully confirmed), but the claimed high severity is not justified ? the vulnerable path is behind an off-by-default feature flag, the cited trigger scenario (delete-to-Trash) doesn't actually invalidate the message id, and the wedge self-heals via the history-expiry fallback with no realistic permanent data loss. Adjusted to medium with confidence 0.8.